

SE 2226 Software Quality Assurance and Testing

Test Plan

Yemeksepeti and Library-DB-Express Application Testing

Prepared by

Atakan Sezginer – 24070006126

Batuhan Yerebasmaz – 23070006029

Ege Çınar – 24070006020

Osman Şahin Güler – 24070006009

İsmet Saygın Koç – 23070006038

19.04.2026

1. Context of Testing

1.1 Project / Test Levels / Test Types

This project covers two complementary test sub-projects carried out within the SE 2226 Software Quality Assurance and Testing course. The first sub-project applies black-box testing techniques to the Yemeksepeti web and mobile application, evaluating the system from the end-user perspective without access to internal source code. The second sub-project applies white-box testing techniques to the Library-DB-Express application (<https://github.com/jlayher/Library-DB-Express>), a RESTful API built with Node.js and Express that manages a library database covering books, authors, members, and loans, using its fully accessible source code to examine internal logic and code coverage.

For the Yemeksepeti sub-project, testing is performed at the System Testing level using Functional Testing and UI Testing as test types following black-box methods. For the Library-DB-Express sub-project, testing is performed at the Unit Testing level using Statement Coverage, Branch Coverage, Basis Path Testing, and Data-Flow Testing as test types following white-box structural methods, supported by static code analysis.

1.2 Test Items

Sub-Project 1 – Yemeksepeti (Black-Box):

The test items are the Yemeksepeti Web Application and the Yemeksepeti Mobile Application. These applications allow users to search for restaurants, view menus, add food items to a cart, and complete an order. Main components to be tested include:

- User account features (login and registration)
- Restaurant browsing and search functionality
- Viewing restaurant menus
- Cart functionality (adding, removing, and changing item quantity)
- Checkout process and order confirmation

Sub-Project 2 – Library-DB-Express (White-Box):

The test items are the source code modules of the Library-DB-Express repository. White-box tests are written in Java using JUnit6 within IntelliJ IDEA Community Edition. Version: latest commit on the main branch at the time of testing. The primary modules under test include:

- app.js / server.js – Express application entry point and middleware configuration
- routes/ – Route handler functions for books, authors, members, and loans endpoints
- db/ – Database connection module and SQL query functions
- middleware/ – Input validation and error-handling middleware

1.3 Test Scope

Sub-Project 1 – Yemeksepeti: Features to be tested:

- User registration and login functionality
- Searching restaurants by name or category
- Viewing restaurant menus
- Adding, removing, and changing item quantity in the cart
- Entering delivery address and completing the checkout process
- Order confirmation behavior

Sub-Project 2 – Library-DB-Express: Features to be tested:

- All conditional branches in route handler functions (if/else, ternary, switch)
- SQL query construction logic (parameter binding, WHERE clause building)
- Error propagation paths: database errors, validation errors, and 404 handling

- Input validation logic for required fields (title, author, ISBN, member ID)
- Business logic for loan availability checks (book already on active loan)

Features not included in testing (both sub-projects):

- Performance testing, load testing, and stress testing (both sub-projects)
- Security testing (both sub-projects)
- Real payment transactions (Yemeksepeti sub-project only)
- Frontend/UI rendering and third-party library internals (Library-DB-Express)
- Infrastructure and DevOps configuration files (Library-DB-Express)

These features are excluded either because the source code and infrastructure are not accessible (Yemeksepeti), or because they fall outside the structural code boundary or belong to external dependencies (Library-DB-Express).

1.4 Test Basis

For the Yemeksepeti sub-project, test cases are designed based on the observable functionality of the application and general requirements of e-commerce systems. Testing is grounded in user interactions with the interface and expected system behaviour. Techniques such as equivalence partitioning, boundary value analysis, use case testing, and decision table methods are used to generate test cases, with expected behaviour determined according to common usability expectations and typical online ordering workflows.

For the Library-DB-Express sub-project, the primary basis is the source code itself. Control-flow graphs (CFGs) are derived from each handler and service function. Branch coverage criteria follow the structural test design techniques defined in ISO/IEC/IEEE 29119-4. JUnit6 annotations such as `@Test`, `@ParameterizedTest`, and `@RepeatedTest` are used to structure and execute the test cases. The project README and inline code comments serve as supplementary documentation for expected behaviour.

2. Assumptions and Constraints

Testing across both sub-projects is conducted under the following shared and sub-project-specific assumptions and constraints.

- The Yemeksepeti sub-project uses black-box testing methods since the internal source code, database structure, and system architecture are not accessible. Testing focuses only on observable system behaviour through the user interface.
- The Library-DB-Express sub-project has full source code access. White-box tests are written in Java using JUnit6 installed via IntelliJ IDEA Community Edition (org.junit:jupiter:junit-jupiter:6.0.2). A dedicated test database or test fixture is used during testing to ensure isolation.
- Real payment transactions will not be performed in Yemeksepeti testing; only the observable checkout steps will be validated.
- Test results for Yemeksepeti may vary depending on device type, browser version, mobile operating system, and internet connection speed.
- Oracle JDK 8 or any JDK 17+ is installed on each team member's machine. Code coverage is measured using IntelliJ IDEA's built-in coverage tool. Target thresholds: 80% statement coverage and 75% branch coverage.
- All testing activities must be completed before the project presentation scheduled for May 4, 2026.
- Possible risks include application updates during the testing period or tool compatibility issues. In such cases, test cases will be updated and manual testing will be performed if necessary.

3. Test Strategy

3.1 General

The project testing strategy is divided into two complementary approaches that together cover both the observable behaviour and the internal code quality of software systems.

For Yemeksepeti, testing evaluates the system from the end-user perspective using black-box techniques. Test cases are designed based on user interactions and expected system behaviour using equivalence partitioning, boundary value analysis, use case testing, and decision table methods. Both manual and automated approaches are used: Selenium IDE for web automation and Maestro Studio for mobile automation.

For Library-DB-Express, testing examines and exercises the internal code paths of the application. Control-flow graphs are constructed for each handler to identify all independent execution paths. JUnit6 is used to write and execute unit test classes within IntelliJ IDEA Community Edition. Static code analysis is applied using the PMDPlugin and CheckStyle-IDEA plugins, and code coverage is measured using IntelliJ's built-in coverage runner.

3.2 Test Levels

For Yemeksepeti, testing is conducted at the System Testing level, where the application is evaluated as a complete system from the end-user perspective across both web and mobile platforms. Acceptance-level considerations are included as testing focuses on real user workflows. Unit and integration testing are excluded because the source code is not accessible.

For Library-DB-Express, testing is conducted at the Unit Testing level. Individual handler and service functions are tested in isolation using JUnit6 test classes. Test methods annotated with `@Test`, `@ParameterizedTest`, and `@RepeatedTest` are used to cover normal behaviour, edge cases, and repeated execution scenarios. Database-dependent paths are tested using dedicated test fixtures or in-memory data setups.

3.3 Test Types

For Yemeksepeti, the main test type is Functional Testing, which verifies whether the application behaves according to expected user interactions. User Interface (UI) Testing is also performed to ensure visual components and navigation function correctly across web and mobile platforms. Regression Testing is performed by re-executing existing test cases to confirm that previously working features continue to function correctly after any changes.

For Library-DB-Express, the test types applied are Statement Coverage Testing (target 80%), Branch/Decision Coverage Testing (target 75%), Basis Path Testing using McCabe's cyclomatic complexity metric, and Data-Flow Testing to verify definition-use pairs for key variables particularly in the loan availability check and input validation logic. Static Code Analysis is also applied using PMD and CheckStyle to identify code quality issues and enforce coding standards.

3.4 Test Deliverables

The following test deliverables will be produced during the testing process:

- Test Plan (this document)
- Test Case Specifications: detailed test scenarios for both sub-projects
- Automated Test Scripts: Selenium IDE scripts for web testing and Maestro scripts for mobile testing (Yemeksepeti)
- JUnit6 test class files (*Test.java) covering unit-level white-box test cases (Library-DB-Express)

- Control-Flow Graph diagrams and IntelliJ built-in coverage report (Library-DB-Express)
- PMD and CheckStyle static analysis reports (Library-DB-Express)
- Incident Reports: documentation of defects and unexpected behaviours
- Test Status Reports and Test Completion Report

3.5 Test Design Techniques

Black-Box Techniques (Yemeksepeti):

- **Equivalence Partitioning:** divides input data into valid and invalid groups to reduce test cases while maintaining coverage
- **Boundary Value Analysis:** tests input values at the limits of acceptable ranges such as minimum and maximum item quantities
- **Use Case Testing:** evaluates complete user workflows such as login, restaurant search, cart management, and order completion
- **Decision Table Testing:** applied where combinations of conditions lead to different outcomes such as login validation or checkout scenarios

White-Box Techniques (Library-DB-Express):

- **Statement Coverage (SC):** every executable line of code runs at least once; measured using IntelliJ built-in coverage; target 80%
- **Branch / Decision Coverage (BC):** both true and false outcomes of every conditional are exercised; target 75%
- **Basis Path Testing (McCabe):** CFGs are drawn for each handler; cyclomatic complexity $V(G)$ is calculated and one JUnit6 @Test method is written per independent path
- **Data-Flow Testing (DFT):** definition-use (DU) pairs for key variables are identified and exercised, especially in loan availability and validation logic
- **Static Code Analysis:** PMDPlugin and CheckStyle-IDEA are applied within IntelliJ to detect code smells, style violations, and potential defects

3.6 Entry and Exit Criteria

Entry Criteria

- The test plan is completed and approved, and the test environment is ready
- Selenium IDE and Maestro Studio are installed and configured; test accounts for Yemeksepeti are available
- IntelliJ IDEA Community Edition is installed with JUnit6 (org.junit.jupiter:junit-jupiter:6.0.2), PMDPlugin, and CheckStyle-IDEA configured
- Oracle JDK 8 or JDK 17+ is installed; Library-DB-Express repository is cloned and accessible
- Control-flow graphs for all handler functions have been drawn and reviewed by the team

Exit Criteria

- All planned test cases for both sub-projects have been executed
- Main Yemeksepeti workflows (login, search, cart, checkout) have been tested and results are documented
- Statement coverage $\geq 80\%$ and branch coverage $\geq 75\%$ for Library-DB-Express as reported by IntelliJ built-in coverage tool
- PMD and CheckStyle reports have been reviewed and critical violations are resolved or documented
- All identified defects are documented; no critical issues remain unresolved
- Testing may be paused if critical application functions become unavailable and will resume once resolved

3.7 Test Completion Criteria

Testing across both sub-projects is considered complete when all planned test cases have been executed, test results are documented and reviewed, coverage targets for Library-DB-Express are met as reported by IntelliJ's built-in coverage tool, and no critical issues remain unresolved. All testing activities and documentation will be completed before the project presentation scheduled for May 4, 2026.

3.8 Metrics to be Collected

During testing, the following metrics will be collected: total number of test cases executed, number of passed and failed test cases, number of defects identified (categorized as minor, major, or critical), statement coverage percentage, branch coverage percentage, cyclomatic complexity V(G) per handler function, and number of PMD and CheckStyle violations found and resolved. These metrics help assess the quality of both applications and the success of the testing activities.

3.9 Test Data Requirements

For Yemeksepeti, testing requires sample user data such as email addresses and passwords for login and registration scenarios. Address information is needed for the checkout process, and various input values are required to test search functionality and cart operations. Test data represents realistic user behaviour while avoiding sensitive personal information.

For Library-DB-Express, test data is prepared as Java test fixtures within JUnit6 test classes using `@BeforeAll` or `@BeforeEach` setup methods. Test data includes at least 3 book records (one with an active loan), at least 2 member records, and at least 1 active loan record. All test data is synthetic and contains no real personal information.

3.10 Test Environment Requirements

The testing environment includes the following hardware and software components. A personal computer running Windows, macOS, or Linux is required for all testing activities. IntelliJ IDEA Community Edition is the recommended IDE for both sub-projects. A stable internet connection is required to access the Yemeksepeti application and the GitHub repository.

Sub-Project 1 – Yemeksepeti:

- Google Chrome browser with Selenium IDE extension for web application testing
- Maven dependency: `org.seleniumhq.selenium:selenium-java:4.40.0` for Selenium WebDriver
- Physical Android or iOS device with Maestro Studio installed for mobile testing
- Android Platform Tools for USB debugging
(<https://developer.android.com/tools/releases/platform-tools>)

Sub-Project 2 – Library-DB-Express:

- IntelliJ IDEA Community Edition with Oracle JDK 8 or any JDK 17+
- JUnit6 via Maven: `org.junit.jupiter:junit-jupiter:6.0.2` and `org.junit.platform:junit-platform-launcher:6.0.2`
- PMDPlugin and CheckStyle-IDEA plugins installed in IntelliJ for static code analysis
- IntelliJ built-in coverage runner for statement and branch coverage measurement

3.11 Testing Activities and Estimates

The testing process for both sub-projects includes the following activities. These estimates are approximate and may vary depending on test complexity and team coordination.

- Test Planning: defining scope, strategy, and tools for both sub-projects (1 day)
- Test Environment Setup: installing Selenium IDE, Maestro, IntelliJ, JDK, JUnit6, PMD, and CheckStyle (1 day)

- CFG Construction and White-Box Analysis: drawing control-flow graphs and calculating cyclomatic complexity for all handler functions (0.5 day)
- Test Design: black-box test cases using equivalence partitioning, boundary value analysis, use case, and decision table; white-box test cases using basis paths and branch coverage (2–3 days)
- Test Implementation: recording Selenium IDE and Maestro scripts; writing JUnit6 @Test, @ParameterizedTest, and @RepeatedTest methods (3 days)
- Test Execution: running all manual and automated tests; reviewing IntelliJ coverage reports and PMD/CheckStyle results (2 days)
- Incident Reporting: documenting defects identified during testing (performed throughout the testing process)

4. Staffing

4.1 General

The testing activities will be performed by a team of five members with knowledge of software testing, quality assurance concepts, and the tools used in this course. Team members collaborate in designing test cases, executing tests, and documenting results across both sub-projects. Each member contributes to both black-box and white-box testing activities according to the role assignments below.

4.2 Roles and Responsibilities

Test Lead – Osman Şahin Güler

Responsible for overall test planning and coordination across both sub-projects, final review of all test documents, and ensuring the testing process follows the defined strategy. Also oversees the white-box test plan, reviews control-flow graphs, and approves coverage reports generated by IntelliJ IDEA.

Web Testers / Unit Test Developers – Atakan Sezginer, İsmet Saygın Koç

Responsible for designing, creating, and executing black-box test cases for the Yemeksepeti web application using Selenium IDE and selenium-java. Also responsible for writing JUnit6 unit test classes for Library-DB-Express handler functions using @Test and @ParameterizedTest annotations, and ensuring that branch coverage targets are met as reported by IntelliJ's built-in coverage tool.

Mobile Testers / White-Box Testers – Batuhan Yerebasmaz, Ege Çınar

Responsible for designing, creating, and executing black-box test cases for the Yemeksepeti mobile application using Maestro Studio. Also responsible for writing JUnit6 @RepeatedTest and @ParameterizedTest methods for Library-DB-Express, running PMD and CheckStyle static analysis, and collecting and reviewing coverage metrics from IntelliJ.

All team members will support each other when necessary and collaborate throughout both sub-projects.

4.3 Schedule

Testing activities for both sub-projects will be conducted throughout the semester in alignment with the SE 2226 course syllabus. All activities and documentation will be completed before the project presentation scheduled for May 4, 2026.

Testing Activity	Estimated Time
Test Planning	1 day
Test Environment Setup (Selenium IDE, Maestro, IntelliJ, JDK, JUnit6, PMD, CheckStyle)	1 day
CFG Construction and White-Box Analysis	0.5 day

Test Design (Black-Box and White-Box test cases)	2–3 days
Test Implementation (Selenium, Maestro scripts; JUnit6 @Test, @ParameterizedTest, @RepeatedTest methods)	3 days
Test Execution, Coverage Review (IntelliJ), and PMD/CheckStyle Analysis	2 days
Incident Reporting and Test Completion Report	Throughout testing